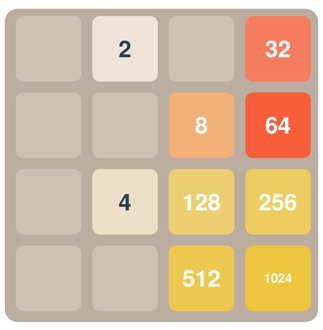
Solving 2048: Classical RL vs. LLM-Guided RLVR

Shriman Raghav[1] Gautham Ramkumar[1]

[1]MS Robotics, Khoury College of Computer Sciences, Northeastern University

CS 5180: Reinforcement Learning | Prof. Robert Platt

Problem: The 2048 Puzzle



Slide tiles in 4 directions. Equal tiles merge, doubling their value. Reach **2048** to win!

Challenges:

- $\sim 10^{19}$ board states
- Sparse, delayed rewards
- Stochastic tile spawns
- 100–1000+ step episodes

States, Actions & Rewards:

$\mathcal{S}$ 4×4 board → 16-channel binary tensor $\in \{0,1\}^{16\times4\times4}$
$\mathcal{A}$ {UP, RIGHT, DOWN, LEFT}: 4 actions
$\mathcal{R}$ Score-delta + milestones (+50@256, +100@512, +200@1024)
γ 0.99

Experiment Setup

- **Hardware**: NVIDIA RTX 4060 (8 GB), CUDA 12.6
- **Training**: 5M env steps per classical agent
- **Agents**: 6 classical + 2 GRPO-LLM (0.5B, 1.5B)
- **Reward**: Shaped (score-delta + milestone bonuses)
- **Eval**: Last 100 episodes + unlimited "Hunt" mode

> **Controlled experiment**: all deep agents share an **identical CNN backbone** (3-layer, 330K params). Differences reflect the *learning algorithm*.

Why custom DQN & SAC? SB3's DQN lacks direct Q-value masking for invalid actions (only action-space wrappers). Our custom PyTorch implementation masks invalid Q-values to $-\infty$ *inside the forward pass*, enabling proper ε-greedy over legal moves only. SAC required a discrete-action adaptation (Christodoulou, 2019) with categorical policies and batched GPU inference for speed.

Algorithms

DQN ★ Best Performer

Custom PyTorch

```
DQN Pseudocode:
1. Sample (s, a, r, s') from B
2. Mask invalid a' to −∞
3. y ← r + γ max_a' Q_θ̄(s', a')
4. L ← SmoothL1(Q_θ(s, a), y)
5. θ ← θ − α∇L
6. Sync θ̄ ← θ every C steps
```

Off-policy, value-based

Buffer:	100K transitions
Batch:	64
ε:	$1.0 \to 0.01$ / 100K
Target:	sync every 1K steps
LR:	10^{-4} (Adam)
Grad clip:	1.0

SAC (Discrete)

Custom PyTorch

```
SAC Pseudocode:
1. Sample (s, a, r, s') from B
2. Mask invalid actions in π
3. y ← r + γ(Q_min − α log π)
4. Update twin Q_1, Q_2 critics
5. Update π via entropy objective
6. Auto-tune α to target entropy
```

Off-policy, max-entropy

Buffer:	200K transitions
Batch:	256
Polyak τ:	0.005
LR:	10^{-4} (Adam)
Warmup:	10K steps
Target ent.:	$0.4 \cdot \log\lvert\mathcal{A}\rvert$

PPO (MaskablePPO)

Stable-Baselines3

```
PPO Pseudocode:
1. Collect rollout (T steps, 8 envs)
2. Compute GAE advantages Â
3. r_t(θ) = π_θ / π_old
4. L = min(r_t Â, clip(r_t) Â)
5. Update θ for K=10 epochs
```

On-policy, actor-critic

Rollout:	2048 steps × 8 envs
Batch:	64
Epochs:	10
LR:	3×10^{-4}
GAE:	λ=0.95
Clip:	ϵ=0.2
Entropy:	c_e=0.01

A2C

Stable-Baselines3

On-policy, actor-critic. Simpler than PPO: no clipping, single gradient step per rollout.

n_steps: 5 LR: 7×10^{-4}
c_e: 0.01 c_v: 0.5
$\mathcal{L} = -\mathbb{E}[\log \pi \hat{A}] + 0.5\|R - V\|^2 - 0.01\mathcal{H}[\pi]$

QR-DQN (Distributional)

Stable-Baselines3

```
QR-DQN Pseudocode:
1. Sample (s, a, r, s') from B
2. For each quantile τ_i:
δ_ij = r + γθ_j(s') − θ_i(s)
3. L = Σ_i ρ^κ_τ̂i(δ_ij)
4. Action ← arg max_a (1/N) Σ_i θ_i
```

Off-policy, distributional

Quantiles:	50 (N=50)
Buffer:	100K
Batch:	64
LR:	10^{-4}
Target:	sync every 1K
ε:	$1.0 \to 0.05$ / 10%

Outputs 4×50=200 values.

Semi-gradient SARSA (S&B Alg. 10.1)

Custom NumPy

```
SARSA Pseudocode (Alg 10.1):
1. Observe s, choose a (ε-greedy)
2. Take action a, observe r, s'
3. Choose a' (ε-greedy from s')
4. δ ← r + γq̂(s', a') − q̂(s, a)
5. w_a ← w_a + αδφ(s)
6. s ← s', a ← a'
```

On-policy, linear FA, no neural net!

$\hat{q}(s,a,\mathbf{w}) = \mathbf{w}_a^\top \phi(s)$
$\mathbf{w} \in \mathbb{R}^{4\times30}$ = **120 params total**

α:	10^{-3}
ε decay:	0.99995/step
$\varepsilon_{\min}$:	0.05

30-dim features (§9.5):
log-tiles, empty ratio, merge potential, 4 monotonicity, corner, smoothness, snake, edge.

> **120 params beats 330K+ param deep nets!** Engineered features provide stronger inductive bias than CNNs (S&B §9.5).

GRPO: Group Relative Policy Optimization

trl + transformers

GRPO = **G**roup **R**elative **P**olicy **O**ptimization (DeepSeek-R1, 2025).
Fine-tunes **Qwen2.5-0.5B / 1.5B** LLMs to play 2048 via verifiable rewards, *without a critic network*. Board state → ASCII text → model generates reasoning + action.

```
GRPO Pseudocode:
1. Sample G completions per prompt
2. Score each with verifier r_i
3. Â_i = (r_i − μ_G)/σ_G
4. L = −Σ_i Â_i log π_θ
−βKL(π_θ || π_ref)
5. Update θ via gradient descent
```

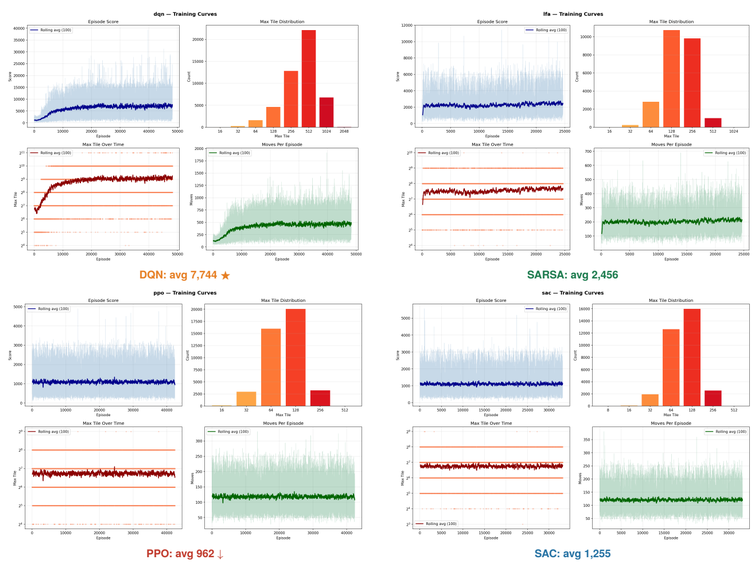
Group-normalized advantage:

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G + \epsilon}$$

G=4 completions per prompt.
3-stage reward scheduling:

1. Format compliance (XML)
2. Game reward (Δscore)
3. Thinking quality

Training Curves (5M Steps)



Results

Final Performance (5M Steps)

Agent	Avg Score	Max Score	Max Tile	512+ Rate	Params	Impl.
DQN ★	7,744	39,314	2048	76%	330K	Custom
SARSA	2,456	11,436	1024	6%	120	NumPy
GRPO 1.5B	2,348	2,348	256	–	1.8B	trl
A2C	1,944	9,640	1024	6%	330K	SB3
GRPO 0.5B	1,816	1,816	128	–	494M	trl
SAC	1,255	7,882	512	<1%	330K	Custom
QR-DQN	995	5,176	512	<1%	430K	SB3
PPO	962	4,900	512	<1%	330K	SB3

Scaling: 1M → 5M Steps

Agent	1M	5M	Δ	Verdict
DQN	2,825	**7,744**	+174%	Strong scaling
SAC	700	1,255	+79%	Slow
A2C	1,244	1,944	+56%	Modest
SARSA	2,299	2,456	+7%	Ceiling (§9.4)
QR-DQN	1,090	995	–9%	Degrading
PPO	1,099	962	–12%	Gets worse!

Classical RL vs. RLVR Comparison

Head-to-Head: Classical vs. LLM-RLVR

Dimension	Classical RL	LLM-RLVR (GRPO)
Best Score	**7,744** (DQN)	2,348 (1.5B)
Best Tile	**2048** (DQN)	256 (1.5B)
Parameters	120 – 430K	**494M – 1.8B**
Inference	**<4 ms / move**	1,500 ms / move
Training data	5M env steps	300 prompts
Interpretable	No	**Yes (CoT)**
Observation	**Tensor (16ch)**	ASCII text
Action space	Numeric	Natural language

Why classical wins on score: DQN processes 5M transitions with replay, while the LLM sees only 300×4=1200 game states. DQN's tensor encoding is information-rich; the LLM must parse text.
Where RLVR excels: *Interpretability*. Each LLM move includes a chain-of-thought explanation that can be inspected, debugged, and corrected. Classical agents are black boxes.
Model scale matters: 0.5B → 1.5B ($3\times$ params) yields $3.3\times$ score improvement, suggesting larger models can better reason about board structure.

Key Findings

1. **Off-policy replay is decisive.** DQN's 100K buffer (S&B §8.2) amplifies sparse rewards. On-policy methods fail because they discard data.

2. **Features > deep nets** (at low data). SARSA (120 params) beats all deep on-policy agents (330K+). Validates S&B §9.5.

3. **More training hurts PPO.** Score drops 12% from 1M→5M. QR-DQN develops invalid-move loops.

4. **GRPO scales with model size.** $3\times$ more params → $3.3\times$ higher score. LLMs provide *interpretable* moves.

Future Work

- Scale GRPO to Qwen2.5-3B / 7B (A100 GPU)
- Scaling curves at checkpoints 50K → 5M
- Multi-seed experiments for error bars
- Hybrid LLM-MCTS exploration

References: [1] Sutton & Barto, *RL: An Introduction*, 2e, MIT Press, 2018. [2] Mnih et al., Nature 518, 2015. [3] Schulman et al., arXiv:1707.06347. [4] Guo et al., DeepSeek-R1, 2025. [5] Shao et al., DeepSeekMath, 2024.